CiT Backend Engineering Track

Week 1: Induction & Orientation

Your Journey into Backend Engineering Starts Here

Detail	Information
Programme	CiT Backend Engineering Track
Phase	Phase 1 — Foundation (Weeks 1–8)
Week	Week 1
Instructor	Jim
Duration	6 months | 24 weeks
Tech Stack	Java 21, Spring Boot, PostgreSQL, Docker

Confidential — For CiT Trainees Only

Opening Prayer

As a Christ-centred community, we begin every session by inviting God's presence, wisdom, and guidance into our learning. Please stand or bow your heads as we pray together.

Prayer of Dedication

Heavenly Father,

We thank You for this opportunity to learn, grow, and develop the skills You have placed within us. We acknowledge that every good and perfect gift comes from You (James 1:17).

As we begin this journey into backend engineering, we ask for Your wisdom and understanding. Open our minds to grasp new concepts, give us patience when things are difficult, and fill us with perseverance to keep going.

Lord, we dedicate this programme to You. May the skills we acquire be used to serve others, to build solutions that make a difference, and to glorify Your name in the technology industry.

Bless our instructors, our fellow trainees, and everyone who has made this training possible. Unite us as a community, and let Your love be the foundation of all we do together.

In Jesus' name we pray,

Amen.

Scripture for the Week

"For I know the plans I have for you," declares the Lord, "plans to prosper you and not to harm you, plans to give you hope and a future."

— Jeremiah 29:11 (NIV)

Table of Contents

1. Welcome to CiT
2. What is Backend Engineering?
3. Setting Up Your Development Environment
4. How the Internet Works
5. Introduction to the Command Line
6. Key Takeaways & Next Steps

1. Welcome to CiT

Welcome to the Centre for Innovators and Technologists (CiT) Backend Engineering Track! Over the next 24 weeks, you will transform from a beginner into a capable backend engineer ready to build real-world applications.

1.1 Programme Overview

This programme is structured into three phases:

- **Phase 1 — Foundation (Weeks 1–8):** Core programming with Java, databases, and engineering fundamentals.
- **Phase 2 — Specialisation (Weeks 9–20):** Spring Boot, API design, security, messaging, caching, and DevOps.
- **Phase 3 — Capstone & Graduation (Weeks 21–24):** Build and present a complete backend project.

1.2 CiT Community Values

As a CiT trainee, you are part of a Christ-centred learning community built on:

- **Faith:** We trust God to guide our learning and careers. We begin every session in prayer and carry that spirit of dependence throughout.
- **Growth Mindset:** Embrace challenges as learning opportunities. Every expert was once a beginner.
- **Collaboration:** Help your peers. Teaching is one of the best ways to learn.
- **Integrity:** Write your own code, ask when you are stuck, and give credit where it is due.
- **Consistency:** Show up, practise daily, and build momentum over time.
- **Professionalism:** Communicate clearly, meet deadlines, and treat every session like a workplace.

1.3 What You Will Build

By the end of this programme, you will have built a complete backend system that includes:

- A Spring Boot REST API with 5+ meaningful endpoints
- A PostgreSQL/MySQL database with a properly designed schema
- JWT-based authentication with multiple user roles
- Asynchronous features using Apache Kafka or scheduled jobs

- Unit and integration tests
- Docker containerisation for easy deployment
- Clean GitHub repository with documentation

Cross-Track Collaboration

You will not be working in isolation! Backend trainees build APIs that Frontend trainees consume. Graphics Design trainees provide Figma designs. Monthly showcases bring all tracks together, and you are encouraged to form multi-track capstone teams.

2. What is Backend Engineering?

When you use a mobile app or website, everything you see and interact with is called the frontend or client side. But behind the scenes, there is a whole system doing the heavy lifting — that is the backend.

2.1 Frontend vs Backend

Aspect	Description
Frontend	What the user sees — buttons, forms, pages, animations
Backend	What the user does not see — data processing, storage, business logic, security

Beginner's Example: Think of a Restaurant

Imagine you walk into a restaurant:

Frontend = The dining area. This is what you see — the menu, the tables, the decor, the plates of food. It is designed to look nice and be easy for you to use.

Backend = The kitchen. This is what you do not see — the chefs cooking your food, the fridge storing ingredients, the recipe book with instructions. Without the kitchen, the restaurant cannot function, even though you never see it.

When you tap "Order" on a food delivery app like Jumia Food or Glovo:

What you see (Frontend): A button that says "Place Order", a loading spinner, then a confirmation message.

What happens behind the scenes (Backend): The server checks if the restaurant is open, verifies your payment, saves the order to a database, notifies the restaurant, and assigns a delivery rider.

The backend does all the heavy work — the frontend just shows you the result!

2.2 What Backend Engineers Do

As a backend engineer, your responsibilities include:

- **Building APIs:** Creating endpoints that frontend applications and other services can call to get or send data.
- **Managing Databases:** Designing how data is stored, retrieved, and updated efficiently.
- **Implementing Business Logic:** Encoding the rules of how a business operates — e.g., a ride-hailing app matching drivers to riders.

- **Handling Authentication & Security:** Ensuring only authorised users can access certain resources.
- **Ensuring Performance & Scalability:** Making sure the system stays fast even as thousands of users use it simultaneously.

2.3 Real-World Examples

Think about these everyday applications and what the backend does:

Mobile Money (e.g., MTN MoMo, Airtel Money)

- Verifies your PIN and account balance
- Processes the transaction and updates both sender and receiver accounts
- Sends an SMS confirmation
- Logs the transaction for auditing

Social Media (e.g., Twitter/X)

- Stores your posts, followers, and profile data
- Determines what appears in your timeline (recommendation algorithm)
- Sends push notifications in real time
- Handles millions of concurrent users

2.4 Why Java & Spring Boot?

We chose Java and Spring Boot because:

1. Java is the most widely used backend language in enterprise and fintech companies across Africa and globally.
2. Spring Boot makes it fast to build production-ready APIs with minimal boilerplate.
3. The Java ecosystem has mature libraries, tooling, and a massive community for support.
4. Java skills are in high demand — banks, telecoms, and tech companies actively hire Java backend engineers.

Exercise 1: Backend Thinking

Answer the following questions in your notebook or a text file. Discuss your answers with a partner.

Question 1:

Pick any app you use daily (e.g., WhatsApp, Uber, YouTube, Jumia). List at least 5 things the backend must do when you perform a common action in that app. For example, what happens on the backend when you send a WhatsApp message?

Question 2:

A hospital wants to build a patient management system. What kind of data would the backend need to store? What business rules would you need to implement? Think about appointments, prescriptions, and billing.

Question 3:

For each of the following features, identify whether it is a frontend or backend responsibility (or both):

- Displaying a user's profile photo
- Checking if a username is already taken during registration
- Sending a password reset email
- Showing a loading spinner while data is being fetched
- Calculating the total price of items in a shopping cart
- Encrypting a user's password before storing it

3. Setting Up Your Development Environment

Before writing any code, you need to set up the tools of the trade. Think of these as your workshop — a carpenter needs a workbench, saw, and measuring tape. You need:

3.1 JDK 21 (Java Development Kit)

The JDK is what allows your computer to compile and run Java programmes. We use **JDK 21** because it is the latest Long-Term Support (LTS) version.

Installation Steps:

1. Visit https://adoptium.net/ (Eclipse Temurin is recommended)
2. Download the installer for your operating system (Windows, macOS, or Linux)
3. Run the installer and follow the prompts
4. Verify the installation by opening a terminal and typing:

```
java --version
```

You should see output showing Java 21.x.x

Alternative: Install Java Using SDKMAN (Recommended for macOS/Linux)

SDKMAN is a tool that makes installing and managing Java versions very easy. It works on macOS, Linux, and Windows (via Git Bash or WSL).

Step 1: Install SDKMAN

Open your terminal and paste this command:

```
$ curl -s "https://get.sdkman.io" | bash
```

Then open a new terminal window, or run:

```
$ source "$HOME/.sdkman/bin/sdkman-init.sh"
```

Step 2: Verify SDKMAN is installed

```
$ sdk version
SDKMAN! 5.18.2
```

Step 3: Install Java 21

```
$ sdk install java 21.0.4-tem
```

This installs Eclipse Temurin JDK 21 automatically. No downloading or clicking needed!

Step 4: Verify Java is installed

```
$ java --version
openjdk 21.0.4 2024-07-16
```

Why SDKMAN?

- One command to install Java — no need to visit websites or run installers
- Easily switch between Java versions if needed
- Works the same on macOS and Linux

We recommend using SDKMAN as it is what professional developers use to manage their Java installations.

3.2 IntelliJ IDEA Community Edition

IntelliJ IDEA is the most popular IDE (Integrated Development Environment) for Java development. The Community Edition is free and has everything you need.

Installation Steps:

1. Visit https://www.jetbrains.com/idea/download/
2. Download the Community Edition (free) for your OS
3. Install and launch IntelliJ
4. On first launch, configure the JDK: Go to File > Project Structure > SDKs and point it to your JDK 21 installation

Tip

IntelliJ has powerful features like auto-completion, refactoring tools, and a built-in debugger. Take time to explore the interface — it will become your primary workspace.

3.3 Git & GitHub

Git is a version control system that tracks changes in your code. **GitHub** is a platform where you store your Git repositories online and collaborate with others.

Why version control matters:

- You can undo mistakes by going back to a previous version
- Multiple people can work on the same codebase without conflicts
- Your entire project history is preserved
- It is an industry standard — every professional developer uses Git

Setup:

5. Install Git from https://git-scm.com/downloads
6. Create a GitHub account at https://github.com if you do not have one
7. Configure Git with your name and email:

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your.email@example.com"
```

3.4 Postman

Postman is a tool for testing APIs. You will use it extensively to send requests to your backend and inspect the responses.

- Download from https://www.postman.com/downloads/
- Create a free account
- We will use Postman heavily from Week 9 onwards when you start building APIs

4. How the Internet Works

To build backend systems, you need to understand how computers communicate over the internet. This section breaks it down into simple concepts.

4.1 Clients and Servers

The internet works on a **client-server model**:

- **Client:** The device or application making a request (your browser, a mobile app, Postman).
- **Server:** The computer that receives the request, processes it, and sends back a response.

When you type www.google.com in your browser:

5. Your browser (client) sends a request to Google's server
6. The server processes the request
7. The server sends back the Google homepage (response)
8. Your browser displays the page

Beginner's Example: Client-Server is Like a Phone Call

Think of calling Safaricom or MTN customer care:

You (Client): You dial the number and say, "I want to check my data balance."

Customer Care Agent (Server): They look up your account, check the balance, and tell you, "You have 2GB remaining."

That is exactly how the internet works:

1. Your phone (client) sends a request: "Show me my Instagram feed."
2. Instagram's server receives the request and looks up your data.
3. The server sends back your feed (photos, stories, reels).
4. Your phone displays it on screen.

Every time you open an app or website, your device is making a "phone call" to a server somewhere in the world!

4.2 HTTP — The Language of the Web

HTTP (HyperText Transfer Protocol) is the protocol (set of rules) that clients and servers use to communicate. Every web request uses HTTP.

HTTP Methods — The Verbs of the Web:

Method	Purpose	Example
GET	Retrieve data	Fetch a list of products
POST	Create new data	Register a new user account
PUT	Update existing data	Update your profile information
DELETE	Remove data	Delete a blog post

Beginner's Example: HTTP Methods with Your Phone Contacts

Think of the contacts app on your phone. Here is how each HTTP method maps to what you already do:

GET — Looking up a contact: You open your contacts and search for "Mum". You are GETTING (reading) information. Nothing changes.

POST — Adding a new contact: You meet someone new and tap "Add Contact" to save their number. You are POSTING (creating) new data.

PUT — Updating a contact: Your friend changes their phone number, so you edit their contact. You are PUTTING (updating) existing data.

DELETE — Deleting a contact: You remove an old contact you no longer need. You are DELETING data.

These four actions — read, create, update, delete — are the foundation of almost every application!

4.3 HTTP Status Codes

When a server responds, it includes a status code to indicate what happened:

Code	Meaning	When You See It
200	OK	Request was successful
201	Created	A new resource was created
400	Bad Request	You sent invalid data
401	Unauthorised	You need to log in first
404	Not Found	The resource does not exist
500	Internal Server Error	Something went wrong on the server

Beginner's Example: Status Codes in Everyday Life

Think of status codes as the kind of reply you get when you ask for something:

200 (OK): You ask a shopkeeper for bread. They hand it to you. "Here you go!" — everything went well.

201 (Created): You go to the bank and open a new account. "Your account has been created successfully!"

400 (Bad Request): You walk into a restaurant and ask for "23 kilos of air." The waiter says, "That does not make sense." Your request was badly formed.

401 (Unauthorised): You try to enter a VIP lounge without a ticket. The bouncer says, "You need a pass to enter."

404 (Not Found): You go to a shop looking for a specific product, but the shopkeeper says, "We do not have that item." It does not exist here.

500 (Server Error): You order food at a restaurant, but the kitchen catches fire. The waiter comes back and says, "Sorry, something went wrong on our side."

Tip: You have probably seen "404 Not Found" when you click a broken link on a website — now you know what it means!

4.4 What is an API?

An **API (Application Programming Interface)** is a set of rules and endpoints that allow different software applications to communicate with each other.

Think of an API like a waiter in a restaurant:

- You (the client) tell the waiter what you want
- The waiter takes your order to the kitchen (the server)
- The kitchen prepares your food (processes the request)
- The waiter brings the food back to you (the response)

As a backend engineer, you will be building APIs — the "menu" that tells other applications what services are available and how to use them.

Beginner's Example: APIs in Action — Sending Mobile Money

Let's walk through what happens when you send money using MTN MoMo:

1. You open the MoMo app and enter the recipient's number, amount, and your PIN.
2. The app (client) sends this information to MTN's server through an API.
3. The API is like a set of rules that says: "To send money, you must provide: sender number, receiver number, amount, and PIN."

4. The server checks your PIN, checks your balance, moves the money, and sends back a response.

5. The app shows you: “Transaction successful! You sent UGX 10,000 to 0771234567.”

Without the API, the app would have no way to talk to MTN’s server. The API is the bridge between what you see on screen and what happens on the server.

As a backend engineer, YOU will be the one building these APIs!

4.5 JSON — The Language APIs Speak

JSON (JavaScript Object Notation) is the most common format for sending and receiving data through APIs. It is human-readable and easy for computers to parse.

Example — A user profile in JSON:

```
{
  "name": "Amara Okafor",
  "email": "amara@example.com",
  "age": 24,
  "isStudent": true
}
```

Beginner’s Example: Breaking Down JSON Step by Step

JSON can look confusing at first. Let’s break it down piece by piece using a simple example — a boda boda ride:

```
{
  "rider": "Grace Achieng",
  "pickup": "Wandegeya",
  "destination": "Makerere University",
  "fare": 3000,
  "paid": false
}
```

Here is what each part means:

Curly braces { } — These wrap the entire object. Think of them as a container or envelope.

Keys (left side) — These are labels like "rider", "pickup", "fare". They describe what the data is. Always in double quotes.

Values (right side) — These are the actual data. They can be:

- Text (strings): wrapped in double quotes — "Grace Achieng", "Wandegeya"
- Numbers: written without quotes — 3000
- True/False (booleans): written without quotes — false

Colons : — Separate the key from its value.
Commas , — Separate each key-value pair (except the last one).
Think of JSON as filling out a form — each line is a field (key) and its answer (value)!

Exercise 2: HTTP & APIs

Test your understanding of how the internet works with these exercises.

Part A — Match the HTTP Method:

For each action below, write which HTTP method (GET, POST, PUT, or DELETE) would be used:

9. Viewing your bank account balance
10. Creating a new social media account
11. Changing your profile picture
12. Removing a product from your shopping cart
13. Searching for a book on an online store
14. Submitting a job application form
15. Updating your delivery address
16. Unsubscribing from a mailing list

Part B — Status Code Scenarios:

What HTTP status code would the server return in each scenario?

5. You successfully log in to your account
6. You try to access a page that has been removed
7. You submit a form but forget to fill in required fields
8. The server crashes while processing your request
9. You try to view someone else's private messages without being logged in
10. You successfully register a new account

Part C — Write JSON:

Write a JSON object for each of the following. Use proper JSON syntax with double quotes and correct data types:

8. A book with title, author, year published, number of pages, and whether it is available
9. A student with name, registration number, course, year of study, and a list of 3 subjects

10. A mobile money transaction with sender phone number, receiver phone number, amount, currency, and timestamp

5. Introduction to the Command Line

The command line (also called terminal, shell, or console) is a text-based interface where you type commands instead of clicking buttons. As a backend engineer, you will use the command line daily.

5.1 Why the Command Line?

- Many server environments have no graphical interface — you can only interact via the command line
- It is faster and more powerful than clicking through menus
- Git, Docker, and many developer tools are command-line first
- It gives you precise control over your computer

5.2 Opening the Terminal

- **macOS:** Open Spotlight (Cmd + Space), type "Terminal", and press Enter
- **Windows:** Search for "Command Prompt" or "Windows Terminal" or "Git Bash" (recommended)
- **Linux:** Press Ctrl + Alt + T or search for "Terminal"

5.3 Essential Commands

Here are the commands you will use most frequently:

Navigating the File System:

Command	What It Does
pwd	Print Working Directory — shows where you are
ls	List files and folders in the current directory
ls -la	List all files including hidden ones, with details
cd foldername	Change directory — move into a folder
cd ..	Go up one level (parent directory)
cd ~	Go to your home directory

Working with Files and Folders:

Command	What It Does
mkdir myproject	Create a new folder called “myproject”
touch hello.txt	Create a new empty file
cat hello.txt	Display the contents of a file
cp file.txt backup.txt	Copy a file
mv file.txt newfolder/	Move a file to another folder
rm file.txt	Delete a file (be careful — no undo!)

Beginner’s Example: A Step-by-Step Terminal Walkthrough

Let’s walk through exactly what happens when you type these commands. Open your terminal and follow along:

Step 1: Where am I?

```
$ pwd
/Users/grace
```

This shows your current location — like asking “What folder am I in right now?”

Step 2: What is in this folder?

```
$ ls
Desktop  Documents  Downloads  Music
```

This lists everything in your current folder, just like opening a file explorer window.

Step 3: Create a new folder

```
$ mkdir my-first-project
$ ls
Desktop  Documents  Downloads  Music  my-first-project
```

You created a new folder! Notice it now appears when you run ls again.

Step 4: Go into the folder

```
$ cd my-first-project
$ pwd
/Users/grace/my-first-project
```

You moved into the new folder. Notice pwd now shows you are inside it.

Step 5: Create a file

```
$ touch hello.txt
$ ls
hello.txt
```

You created an empty file called hello.txt. It now shows up in your folder.

Step 6: Go back up

```
$ cd ..
```

```
$ pwd
/Users/grace
```

The two dots (..) mean "go up one level" — like pressing the back button in a file explorer.

Notice the pattern: you type a command, press Enter, and the terminal responds. That is all there is to it!

Exercise 3: Command Line Practice

Complete each part in your terminal. Take a screenshot of your terminal after each part as proof of completion.

Part A — Getting Started (Basic):

11. Open your terminal
12. Check your current location using `pwd`
13. List all files in your current directory using `ls`
14. Verify your Java installation: `java --version`
15. Verify Git is installed: `git --version`

Part B — Build a Project Structure:

Using only the command line, create the following folder structure:

```
cit-backend/
  src/
    Main.java
  docs/
    notes.txt
  README.txt
```

Hints: You will need `mkdir`, `cd`, and `touch` commands. After creating the structure, navigate to the `cit-backend` folder and run `ls -la` to verify everything is in place.

Part C — File Operations:

11. Navigate into the `docs` folder you created
12. Copy `notes.txt` to a new file called `backup-notes.txt`
13. List the contents of `docs` to confirm both files exist
14. Move `backup-notes.txt` up one level to the `cit-backend` folder
15. Navigate back to `cit-backend` and verify `backup-notes.txt` is there
16. Delete `backup-notes.txt` (careful — no undo!)

Practice Makes Perfect

The command line may feel unfamiliar at first. That is completely normal. The more you use it, the more natural it becomes. Try to use the terminal for everyday file operations this week instead of your file explorer.

Exercise 4: Putting It All Together (Bonus Challenge)

This challenge exercise combines everything you learned in Week 1. Work through it on your own, then discuss with a partner.

Scenario:

You are building the backend for a school management system called "EduTrack". Think through the following:

17. List 5 types of data (resources) this system would need to manage (e.g., students, teachers, etc.)
18. For each resource, write a sample JSON object with at least 4 fields and appropriate data types
19. Design 3 API endpoints for the "Students" resource. For each, specify the HTTP method, a descriptive URL path, and what it does. Example: GET /students — Retrieves a list of all students
20. For each endpoint, what status code would be returned on success? What about on failure?
21. Using the command line, create a project folder structure for EduTrack with folders for: src, docs, tests, and config. Create a README.txt file in the root.

Tip for the Bonus Challenge

There are no wrong answers here — this exercise is about thinking like a backend engineer. The more you practise designing systems on paper before writing code, the better engineer you will become.

6. Key Takeaways & Next Steps

6.1 What You Learned This Week

- Backend engineering is about building the logic, data, and services that power applications
- The client-server model is the foundation of how the internet works
- HTTP provides a standard way for clients and servers to communicate
- APIs are interfaces that let different software talk to each other
- JSON is the common data format for APIs
- The command line is an essential tool for every developer
- You set up your development environment: JDK 21, IntelliJ IDEA, Git, and Postman

6.2 Homework / Self-Study

Complete these tasks before the next session:

17. Ensure all tools are installed and working (JDK 21, IntelliJ, Git, Postman)
18. Create a GitHub account if you have not already, and explore the interface
19. Practise 10 terminal commands from the tables above — try creating folders, files, navigating, and listing contents
20. Read: What is an API? (search for beginner-friendly articles or watch a short YouTube video)

6.3 Looking Ahead — Week 2

Next week, **Victor Kakama** will take you through **Engineering Foundations & Excellence** — the habits, practices, and mindset that separate good engineers from great ones. Come prepared with questions!

Closing Reflection

As we close this session, let us remember that the skills we are learning are tools to serve others and honour God. Colossians 3:23 reminds us:

"Whatever you do, work at it with all your heart, as working for the Lord, not for human masters."

— Colossians 3:23 (NIV)

Go forth and code with purpose!

Glossary of Terms

Term	Definition
API	Application Programming Interface — a set of rules for software to communicate
Backend	The server-side part of an application that handles data and logic
Client	A device or application that sends requests to a server
Git	A version control system that tracks changes in your code
GitHub	An online platform for hosting and collaborating on Git repositories
HTTP	HyperText Transfer Protocol — the rules for web communication
IDE	Integrated Development Environment — a tool for writing code (e.g., IntelliJ)
JDK	Java Development Kit — tools needed to develop and run Java applications
JSON	JavaScript Object Notation — a lightweight data format used by APIs
REST	Representational State Transfer — an architectural style for designing APIs
Server	A computer that receives requests, processes them, and sends responses
Terminal	A text-based interface for interacting with your computer via commands